Effective High-Level Synthesis Design Space Exploration through a Novel Cost Function Formulation

Yiheng Gao and Benjamin Carrion Schafer
Department of Electrical and Computer Engineering
The University of Texas at Dallas, TX, USA
{yiheng.gao, schaferb}@utdallas.edu

2021 IEEE International Symposium on Circuits and Systems (ISCAS) | 978-1-7281-9201-7/20/$31.00 ©2021 IEEE | DOI: 10.1109/ISCAS51556.2021.9401684

***Abstract*—High-Level Synthesis (HLS) allows to synthesize untimed behavioral descriptions into efficient RTL descriptions (Verilog or VHDL). One key advantage of HLS is that it allows to generate different micro-architectures from the same behavioral description by simply specifying different mixes of synthesis options. These include, how to synthesize arrays (e.g. RAM or registers), loops (unroll completely, partially or not unroll or pipeline) and functions (inline or not). Ideally, the designer would like to find a mix that leads to the smallest and fastest micro-architecture. However, these design parameters are conflicting which means that there is not a single optimal solution but a range of solutions that form a Pareto-front. Out of all the possible synthesis combinations, the most important ones are the ones that lead to Pareto-optimal micro-architectures that form this front. Thus, automatic HLS design space exploration methods are desirable to efficiently find these micro-architectures automatically. This work investigates how formulating the cost function affects the quality of the exploration and its runtime and proposes a new cost function that leads to better results compared to the traditional formulation based on weighted-sum of the design metrics. We show that the newly proposed cost function leads to better results for three well-known meta-heuristics. In particular simulated annealing, genetic algorithm and ant-colony optimization.**

I. Introduction

High-Level Synthesis (HLS) is being gradually adopted by most VLSI design companies, mainly to design their dedicated hardware accelerators. These are often initially specified algorithmically using traditional software programming languages like ANSI-C, C++ or Matlab, and hence, it makes sense to have a direct way to synthesize these into RTL.

HLS can be defined as the synthesis process that converts untimed behavioral descriptions into efficient RTL code (e.g. Verilog or VHDL). Among many advantages over the use traditional use of low-level hardware description languages (HDLs), one that has proven extremely powerful is the ability of generate micro-architectures with unique design metrics without the need to modify the behavioral description. In most commercial HLS tools, this is typically done by using synthesis directives in the form of pragmas that are annotated in the source code as comments. This allows to e.g. control how to synthesize arrays (e.g. RAM or Registers), loops (e.g. fully unroll, partially unroll, not unroll or pipeline) and functions (e.g. inline or not). These pragmas enable the re-use of the behavioral description in future project that require different design constraints only requiring to specify a new mix of the synthesis directives. Moreover, a designer might want to study all possible micro-architecture candidates before choosing the one that better fits the project needs. Thus, a design space exploration (DSE) on the behavioral description

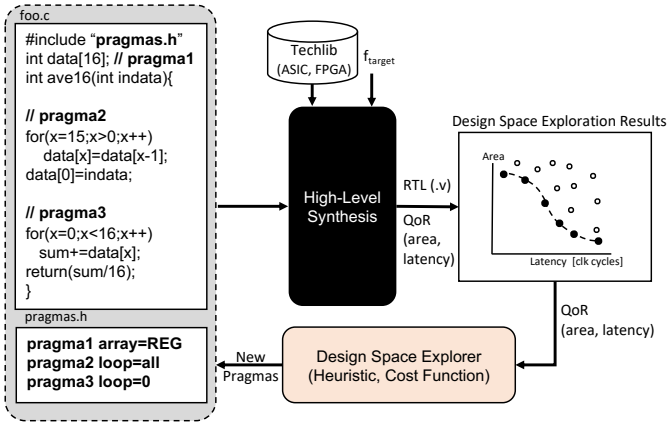

Fig. 1. HLS DSE overview highlighting using the example of the moving average of 16 numbers as example.

can be executed. Out of all the synthesized designs, the most important ones are the ones that form the Pareto-front. Pareto-optimal means that no other design in the search space has simultaneously less area (A) and less latency (L) (in case that only these two design metrics are considered).

HLS DSE can be classified as a multi-objective optimization problem as the main goal is to minimize a set of conflicting design parameters like area and latency. One of the main problems with HLS DSE that the total number of synthesis options combinations is extremely large. Especially considering that, the HLS tool needs to be invoked for every new combination to evaluate its effect on the design metrics. This makes an exhaustive enumeration of all combinations impossible.

Fig. 1 show a typical HLS Design Space Exploration (DSE) flow. The input is an untimed behavioral description (in this case a function that computes the moving average of 16 numbers), the synthesis directives specified in a header file, a target synthesis frequency and a technology library. The C code has one array and two loops. These statements are considered *explorable* because they can be controlled by a synthesis directives as shown. In this example the array is synthesized as registers, the first loop fully unrolled and the second loop not unrolled. Invoking the HLS tool with this mix of pragmas leads to the generation of a particular micro-architecture with specific area and latency. The main goal of the explorer is to calculate the cost of this new solution and to generate a new set of synthesis directives that minimizes this cost function.

Thus, numerous heuristics have been proposed to address this problem. Some of the proposed heuristics are based on well-known meta-heuristics. Meta-heuristics are problem inde-

978-1-7281-9201-7/21/$31.00 ©2021 IEEE
Authorized licensed use limited to: Northwestern University. Downloaded on September 29,2025 at 18:03:28 UTC from IEEE Xplore. Restrictions apply.

pendent heuristics that have shown to lead to very good results for multi-objective optimization problems like this one. Meta-heuristics are often naturally inspired like simulated annealing (SA), genetic algorithm (GA), and ant colony optimizations (ACO).

The traditional way to formulate the cost function in these heuristics is through a weighted-sum that. This sum projects the different design metrics considered by the explorer into a single numeric value that can then be used as cost In this work we only focus on the area (A) and latency (L) trade-offs, therefore the weighted-sum cost function is given as:

$$Cost = \alpha \cdot \frac{A}{A_{max}} + \beta \cdot \frac{L}{L_{max}} \quad (1)$$

where α and β represent the importance of exploring the search space that minimizes the area or the latency , and $\alpha + \beta = 1$. The area (A) and latency (L) are the design metrics from the last synthesized design. These metrics need to be normalized to make up for the unit differences between the cost function elements. This is typically done by dividing this by the largest area (A_{max}) and latency (L_{max}) found so far.

The cost function dynamically changes based on the search space that wants to be explored (either small area, small latency or a mixed of both). The values of α, β are dynamically updated to fully explore the search space when a given exit condition is reached, e.g. N designs that consecutively do not improve the cost function are generated. The step at which these weights are updated (Δ_{cost}) significantly impacts the quality of the exploration results and the running time. Thus, often only three combinations are used: $[\alpha, \beta] = \{[1, 0], [0.5, 0.5], [0, 1]\}$ (Δ_{cost}=0.5)

The main disadvantage of this weighted-sum cost function formulation is that it is hard to set the Δ_{cost}. We have consistently seen that the explorer misses some Pareto-optimal designs (compared to an exhaustive search, which is possible for smaller benchmarks), even setting Δ_{cost}=0.1. Based on this it is important to study how different cost function formulations can help exploring the search space more efficiently. In summary, the main contributions of this work are:

- Investigate the quality of results of the traditional weighted-sum cost function in the context of HLS DSE for multiple meta-heuristics.
- Present a new cost functional formulation that improves the results obtained by the weighted-sum cost function.

II. Related Work

The raise of the level of abstraction to C-based VLSI design and its ability to generate micro-architectures with different trade-offs has led to the publication of numerous work in the filed of HLS design space exploration (DSE). One of the main challenges in HLS DSE is that the search space grows exponentially with the number of explorable operations (array, loops and functions). Numerous heuristics have been therefore proposed to efficiently search the space in order to find the Pareto-optimal configurations. In a recent survey we summarized the most salient work in this domain [1]. The authors classify the heuristics in synthesis based and model based, where the first family of methods need to synthesize any new pragma combinations [2], [3], [4], [5], [6], [7], while the second generate a predictive model to minimize the number of times that that HLS process needs to be invoked [8], [9] . Some more recent work even claim to be able to do a full HLS DSE by creating an offline predictive model a by simply analyzing the CDFG for the application to be explored [10], [11]. These methods nevertheless only target FPGAs as the search space is much smaller.

Our work is fully orthogonal to this previous work as we do not aim at introducing any new heuristic, but a new cost function declaration to perform a more efficient exploration. Our proposed cost function can be used in all of this previous work that mostly use the weight-sum cost function.

III. Meta-Heuristics for HLS DSE

This section introduces how SA, GA and ACO can be used in the context of HLS DSE. Due to the space limitations only a very brief overview of the three methods is possible. In all methods, the *explorable* operation $OP = \{loops, arrays, func\}$, where $loops = \{unroll_all, partial, not_unroll, fold\}$, $arrays = \{RAM, register, ROM, logic\}$ (ROM and logic only for read only arrays) and $func = \{inline, goto\}$.

Simulated Annealing for HLS DSE: Simulated Annealing was initially proposed by Kirpatrick et al.[12] as an effective heuristic for multi-objective optimization problems which is similar to our problem. It refers is based on an analogy of how metals cool and anneal, but instead of using the energy of a material it uses a cost function. Since then, different kinds of optimizations have been implemented upon the basic simulated annealing. The salient idea in SA is that new designs with lower costs are always accepted as a new solution, while new designs with larger costs are still accepted based on the given probability function:

$$Prob_{accept_worse} = 1 - exp^{\frac{\Delta_{Cost}}{T}} \quad (2)$$

where T represents the annealing temperature and Δ_{Cost}= $Cost_{design}$-$Cost_{new_design}$. It can be observed that the probability of accepting a worse solution decreases with the cost difference and increases with a higher temperature. This implies that when the SA explorer starts, the probability of accepting the worst solutions is higher and as it continues and the temperature is reduced, this probability falls.

Genetic Algorithm for HLS DSE: In GA every unique knob set is a gene and all different knobs set to unique values built a chromosome Cr. This Cr is then combined and mutated based on pre-defined crossover and mutation probabilities (pc and pm) to produce an offspring. Thus, the explorer starts by generating two parents with random Cr and synthesizing these in order to obtain their area, performance and power. The cost of each parent can in turn be computed from this.

Given the mutation and crossover rate, an offspring with a unique set of attributes is generated, and the explorer

Authorized licensed use limited to: Northwestern University. Downloaded on September 29,2025 at 18:03:28 UTC from IEEE Xplore. Restrictions apply.

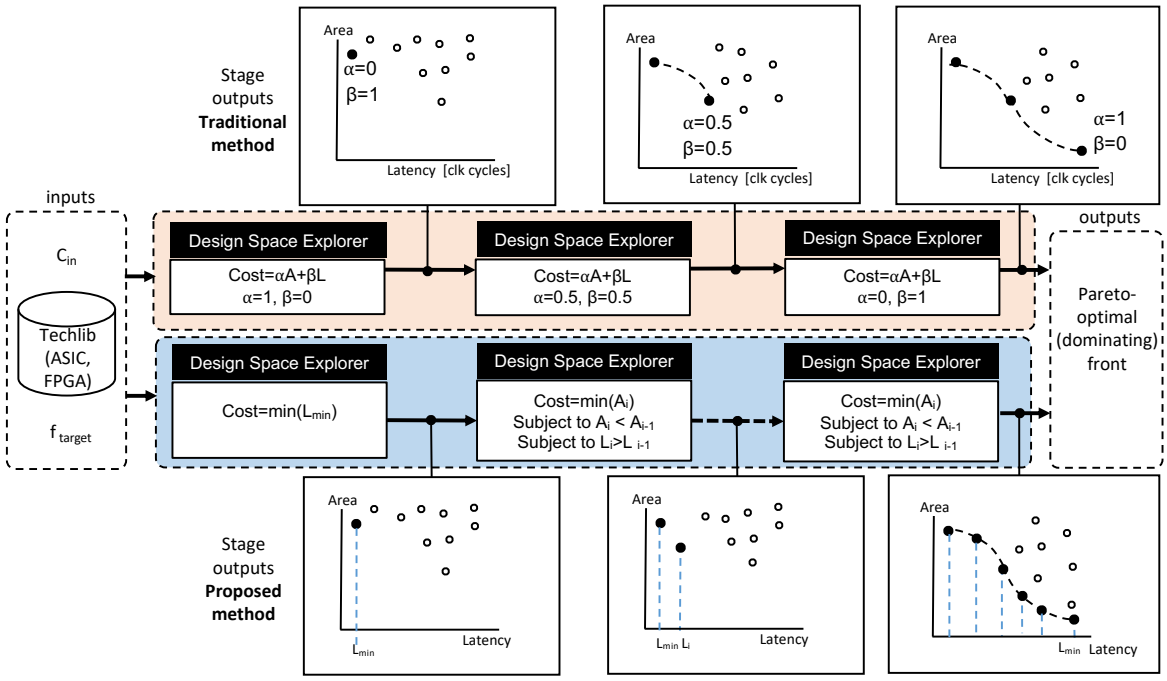

Fig. 2. Overview the effect of the cost function on the exploration results for the traditional method and our proposed method.

synthesizes it. The explorer then continues by comparing the result of the child vs. its parents and substitutes one of them in case that any of the child has a lower cost. This process is repeated until no child can improve the cost function for N iterations, where N is a GA specific hyper-parameter.

Ant Colony Optimization for HLS DSE: ACO was originally introduced in [13] to solve the traveling salesman problem (TSP). It is based on the behavior of ants when they look for food. Initially, all ants leave the nest in random directions and leave a trace of *pheromone* behind. Ants that successfully find food faster than the others return to the nest quicker and intensify the pheromone trace. Other ants in turn will follow the strongest pheromone trace, thus, in turn further intensifying this trail.

In order to solve the TSP problem, ACO associates a pheromone path τ_{ij} for each edge (i, j). The pheromone level indicates the attractiveness of the edge and serves as a distributed global heuristic. Initially, τ_{ij} is set with some fixed value τ_0. For each iteration, N ants are released randomly on the cities, and each starts to construct a path. In order to guarantee that the constructed path is Hamiltonian, every ant will have memory about the cities it has visited so far. If at step t the ant is at city i, the ant chooses the next city j probabilistically using the following expression:

$$P_{ij} = \frac{\tau_{ij}(t)^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{l \in N_i} (\tau_{il}(t)^{\alpha} \cdot \eta_{il}^{\beta})} \tag{3}$$

where N_i represents all possible paths for the ant at vertex i, η_{ij} is a problem-related heuristic (also called local heuristic) calculated every time a probability is generated and α and β are weights to control the influence of the global and local heuristic, respectively.

Based on this formula, there is a higher probability that ants will choose cities with higher pheromone levels and better local heuristic. The pheromone levels have to be constantly updated with an evaporation ratio ρ) within [0,1], which controls the speed at which the pheromone is reduced.

In our case, each explorable operation OP_i is represented as a node which the ant needs traverse to which a synthesis attribute $attr$ is assigned. Each OP_i has a list of attributes from which to choose from. Each attribute (pragma) has assigned a pheromone level τ, which helps the ant decided which attribute $\in OP_i$ to choose for each operation.

IV. Proposed Method Overview

Fig. 2 summarizes the proposed flow, and compares it against a traditional HLS DSE approach using weighted-sum method with Δ_{Cost}=0.5.

The proposed exploration method starts by setting the cost function in oder to find the micro-architecture with the smallest latency, but largest area $[A_{min1}, L_{min}]$. This is shown at the output of the first invocation of the explorer in Fig. 1 . The explorer continues by setting the ξ constraint to L_{min} and re-running the explorer. This basically shrinks the search space to discover (A_{min2}, L_2), where $A_{min2} > A_{min1}$ and $L_2 < L_{min}$. The process then iterated until no new design is found that meets this constraint. This method ensures that the entire search space is swept across.

One critical parameter now is how to decide when the smallest design for a given latency constraint has been found? This can be done by exiting the search when after N consecutive designs no smaller circuit is found. A large N should lead to better results, albeit leading to longer runtimes. In our case we found empirically that using N=5 leads to a good compromise between quality of results and explorer running time. In ξ-`Constraint` the cost function is formulated as:

Authorized licensed use limited to: Northwestern University. Downloaded on September 29,2025 at 18:03:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ξ-CONSTRAINT VS. WEIGHTED-SUM CONSTRAINT RESULTS COMPARISON FOR THREE DIFFERENT META-HEURISTICS (SA, GA AND ACO)

Simulated Annealing (SA) Results

Benchmark	ξ-Constraint SA		Weighted Sum SA	
	ADRS[%]	Run [min]	ADRS(%)	Run[min]
ave16	0.00	3.43	3.1646	3.16
kasumi	6.35	19.93	13.9364	15.17
interpolation	0.00	6.87	0.0000	6.41
gfilter	4.98	57.26	18.6415	50.06
aes	0.25	125.96	8.6528	86.86
t snow3G	0.00	56.22	0.0000	45.38
Avg.	**1.94**		**7.40**	
Geomean		**25.03**		**19.82**
Δ	**3.82**×	**-1.26**×		

Genetic Algorithm (GA) Results

Benchmark	ξ-Constraint GA		Weighted Sum GA	
	ADRS[%]	Run [min]	ADRS(%)	Run[min]
ave16	1.29	2.87	2.4803	2.58
kasumi	0.00	18.32	16.9305	12.35
interpolation	0.00	4.02	0.0000	4.10
gfilter	0.00	60.40	3.0037	40.87
aes	1.69	70.02	8.4791	67.15
snow3G	0.00	40.03	0.0000	33.84
Avg.	**0.50**		**5.15**	
Geomean		**18.15**		**15.16**
Δ	**10.28**×	**-1.20**×		

Ant Colony Optimization (ACO) Results

Benchmark	ξ-Constraint ACO		Weighted Sum ACO	
	ADRS[%]	Run [min]	ADRS(%)	Run[min]
ave16	0.59	3.93	2.46	2.85
kasumi	1.15	21.97	16.93	18.88
interpolation	0.00	11.86	0.00	6.79
gfilter	0.00	70.45	3.66	57.57
aes	2.85	142.54	9.12	117.20
snow3G	0.00	57.18	0.00	50.62
Avg.	**0.76**		**5.36**	
Geomean		**28.94**		**22.35**
Δ	**7.02**×	**-1.29**×		

$$\begin{aligned} &\textbf{Minimize } Cost(A_i), \\ &\textbf{Subject to } L_{i-1} < L_i, \\ &\textbf{Subject to } A_i < A_{i-1} \end{aligned} \tag{4}$$

where A_i and L_i are the area and latency of the new Pareto-optimal point being searched and A_{i-1} and $L_{i=-1}$ the Pareto-optimal point found in the previous exploration round. It should be noted that because heuristics cannot claim to find the optimal result, often in the context of HLS DSE, the best results are referred to as dominating or non-dominated. t

V. EXPERIMENTAL RESULTS

Six computationally intensive applications from the open source S2Cbench SystemC benchmarks suite [14], where used to test our proposed flow. The HLS tools used is NEC's CyberWorkBench. The target technology 45nm nangate open source cell and the target frequency 100MHz. The three meta-heuristics described previously (SA, GA and ACO) where implemented in ANSI-C and have the same exit conditions.

Two set of experiments were conducted. The first compares all three meta-heuristics using the traditional weighted-sum cost function using Δ_{cost}=0.5 (typical method) against our proposed formulation. The goals is to show the robustness of our cost function formulation across different benchmarks and heuristics. The second set of experiments, further studies the efficiency of our proposed cost function in terms of qualify of results vs. runtime against the weighted-sum method when the Δ_{cost} changes from 0.5 to 0.25 to 0.1.

TABLE II
RESULTS COMPARISON AMONG SA, GA AND ACO FOR DIFFERENT WEIGHTED-SUM COST FUNCTION STEPS VS. PROPOSED METHOD.

Method	Algorithm	Avg. ADRS [%]	Geomean Runtime [min]
Weighted-sum (3 steps) Δ_{Cost} =0.5	SA	7.40	19.82
	GA	5.15	15.16
	ACO	5.36	22.35
Weighted-sum (6 steps) Δ_{Cost} =0.20	SA	2.20	33.03
	GA	1.71	23.12
	ACO	1.90	34.11
Weighted-sum (11 steps) Δ_{Cost} =0.1	SA	1.83	77.87
	GA	0.30	59.33
	ACO	0.77	79.83
ξ-Constraint	SA	1.94	25.03
	GA	0.50	18.15
	ACO	0.76	28.94

Table I summarizes the results for the first set of experiments. Average Distance to Reference Set (ADRS) is used to compute the quality of the exploration results. The smaller the ADRS is the better as it indicates that the approximated front is closer to the optimal front. For smaller benchmarks an exhaustive search is used to find the optimal results, while for larger benchmarks the results of all the exploration methods are combined to build the reference front (typical in DSE). From the results, it can be observed that our proposed cost function leads to on average 3.82, 10.28 and 7.02× better ADRS in the SA, GA and ACO heuristics respectively. In terms of runtime, our method is as expected slower, especially considering that in these cases the weighted-sum cost function is only updated three times. The running times are 1.26, 1.20 and 1.29×slower for the SA, GA and ACO respectively. We believe that the extra running time is acceptable considering that the DSE is only executed once and the benefits in the quality of the exploration results obtained.

Table II summarizes the exploration results of the three heuristics when the weighted-sum Δ_{cost} step changes from 0.5 to 0.20 and 0.1 (3 steps, 6 steps and 11 steps). The results show, as expected, that the weighted-sum method's ADRS improves with a smaller Δ_{cost} , while the running time increases.The weighted-sum of 11 steps (Δ_{cost}=0.1) leads to similar results than our method while being on average across all benchmarks and heuristics 3.01× slower.

This results indicate that our proposed cost function works well and that it outperforms the weighted-sum approach, even for the finest-grain one, while being faster.

VI. CONCLUSIONS

In this work we have proposed a new way to formulate the cost function in HLS design space exploration in order to more effectively search through the complete search space. Experimental results show implementing three different meta-heuristics show that the new cost function formulation leads to a better results as compared to traditional formulation based on the weighted sum of the different design metrics.

Authorized licensed use limited to: Northwestern University. Downloaded on September 29,2025 at 18:03:28 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] B. Carrion Schafer and Z. Wang, "High-level synthesis design space exploration: Past, present and future," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–12, 2019.

[2] B. C. Schafer and K. Wakabayashi, "Design space exploration acceleration through operation clustering," *Trans. Comp.-Aided Des. Integ. Cir. Sys.*, vol. 29, no. 1, pp. 153–157, Jan. 2010.

[3] ——, "Divide and conquer high-level synthesis design space exploration," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 17, no. 3, pp. 29:1–29:19, Jul. 2012.

[4] N. K. Pham, A. K. Singh, A. Kumar, and M. M. A. Khin, "Exploiting loop-array dependencies to accelerate the design space exploration with high level synthesis," in *2015 Design, Automation Test in Europe Conference Exhibition (DATE)*. New York, NY, USA: ACM, 2014.

[5] S. Xydis, C. Skouroumounis, K. Pekmestzi, D. Soudris, and G. Economakos, "Efficient High Level Synthesis Exploration Methodology Combining Exhaustive and Gradient-Based Pruned Searching," in *ISVLSI*, July 2010, pp. 104–109.

[6] G. Palermo, C. Silvano, and V. Zaccaria, "Respir: A response surface-based pareto iterative refinement for application-specific design space exploration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 28, no. 12, pp. 1816–1829, Dec 2009.

[7] S. Xydis, G. Palermo, V. Zaccaria, and C. Silvano, "Spirit: Spectral-aware pareto iterative refinement optimization for supervised high-level synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 1, pp. 155–159, Jan 2015.

[8] B. C. Schafer and K. Wakabayashi, "Machine learning predictive modelling high-level synthesis design space exploration," *IET Computers Digital Techniques*, vol. 6, no. 3, pp. 153–159, May 2012.

[9] H.-Y. Liu and L. Carloni, "On learning-based methods for design-space exploration with high-level synthesis," in *DAC*, May 2013, pp. 1–7.

[10] G. Zhong, A. Prakash, Y. Liang, T. Mitra, and S. Niar, "Lin-analyzer: A high-level performance analysis tool for fpga-based accelerators," in *2016 53nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, June 2016, pp. 1–6.

[11] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, "Comba: A comprehensive model-based analysis framework for high level synthesis of real applications," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, Nov 2017, pp. 430–437.

[12] S. Kirkpatrick, "Optimization by simulated annealing: Quantitative studies," *Journal of Statistical Physics*, vol. 34, no. 5, pp. 975–986, Mar 1984.

[13] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996.

[14] B. C. Schafer and A. Mahapatra, "S2cbench: Synthesizable systemc benchmark suite for high-level synthesis," *IEEE Embedded Systems Letters*, vol. 6, no. 3, pp. 53–56, Sept 2014.

Authorized licensed use limited to: Northwestern University. Downloaded on September 29,2025 at 18:03:28 UTC from IEEE Xplore. Restrictions apply.